Allow [[nodiscard]] in type alias declarations

Document #:	P3245R1
Date:	2024-07-15
Project:	Programming Language C++
Audience:	Evolution Working Group Incubator
Reply-to:	Xavier Bonaventura (BMW)

§ 1 Abstract

This paper proposes to allow the usage of [[nodiscard]] to be used in type alias declarations in the same way that can be done with [[deprecated]].

§ 2 Revision history

§ 2.1 R1

Add results of the voting in St. Lous, expand motivation, and add examples of corner cases.

§ 2.2 R0

Initial version

§ 3 State of the art

§ 3.1 [[nodiscard]] in types (currently supported)

[[nodiscard]] was initially proposed in [P0068R0] and [P0189R1] to be used in types and functions. For types, it can be used in the following way:

```
struct [[nodiscard]] Error{
    int code;
};

Error critical_call(){
    return {42};
}
```

```
int main(){
    // Most compilers will issue diagnostics on the next call
    // for any function that returns the Error type
    critical_call();
}
```

Compiler explorer

This allows users to notify the implementation that any function returning a type `Error`, its return value should not be discarded silently.

§ 3.2 `[[nodiscard]]` in type aliases (currently NOT supported)

The addition of `[[nodiscard]]` in a type alias is not allowed by the standard:

```
using MyError [[nodiscard]] = Error;
```

The attribute above will be ignored. The grammer allows the attribute to be in such a position but it is not allowed by the `[[nodiscard]]` attribute specification 9.12.10 [dcl.attr.nodiscard].

At first, one can think that this is not so different than adding the `[[deprecated]]` attribute in a type alias:

```
using OldError [[deprecated]] = Error;
```

But there is an important difference. The alias declaration is not introducing a new type but just a name. In the case of `[[deprecated]]` what we are saying is that the name is deprecated and should not be used. A compiler can issue a warning every time that sees the name without further analysis. When it comes to `[[nodiscard]]` attribute, `[[nodiscard]]` is a property of the type.

§ 4 Motivation

In this paper I would like to propose to allow the usage of `[[nodiscard]]` in type aliases, but before explaining the proposal I would like to elaborate on the motivation.

§ 4.1 Type reuse

Imagine an external library that defines a type `Error` that we want to use in our library. Because this is conceptually an error type, in a safety critical sytem you might be interested on having this type treated as `[[nodiscard]]`.

If the library already marks the type as `[[nodiscard]]` then everything is good an you can use it directly or using an alias. If that is not the case, then it is not so simple.

If we want to reuse `Error` without having to duplicate the whole class, we can use composition or inheritance.

§ 4.1.1 Composition

We could create a `MyError` type that is marked as `[[nodiscard]]` that contains a member of type `Error` and then delegate all calls to `Error` class. The main issue of this approach is that you would have to duplicate the signature of all `Error` methods and make sure that you are forwarding all information in both directions properly.

§ 4.1.2 Inheritance

We could create a `MyError` type that is marked as `[[nodiscard]]` and inherits from `Error`. In this case we would not have to duplicate all method signatures of `Error` and we would only have to take special care for the constructors. The main disadvantages of this approach are all the ones associated with inheritance.

§ 4.2 `std::expected`

There are multiple guidelines for safety critical systems that require that if a function generates an error, such error should be handled. In C++23 we have `std::expected` that is perfect to communicate a value or an error in projects where exceptions are not an option. Because of that, it would not make sense for a developer to create their own type. But what if `std::expected` is not marked as `[[nodiscard]]` in the implementation of the standard library being used?

A developer cannot expect an implementation to `std::expected` as `[[nodiscard]]`, every implementation is free to choose what they believe is more approriate. Additionally, in Tokyo the library policy [P3201R1] was agreed to not use `[[nodiscard]]` in the specification of the standard library and in St. Louis [P2422R0] was voted with strong concensus in favor to remove the current `[[nodiscard]]` annotations of the specification.

§ 5 Proposal

The proposal is to allow the usage of `[[nodiscard]]` in type aliases. This is already possible in clang in a none standard way using compiler annotations.

Before	After

`// In clang`
`using MyError [[clang::warn_unused_result]]`
`  = Error;`
 	
`// In gcc (not possible without MyError`
`  being a new type)`
	`using MyError`
	`  [[nodiscard]]`
	`  = Error;`

Before	After

`// It has __attribute__((warn_unused_result)) but it is not allowed in type aliases`
 	
`// In MSVC (not possible without MyError being a new type)`
`// It has _Check_return_ but it is not allowed in type aliases`	

Compiler explorer

§ 5.1 Problems and potential solutions

The main problem with trying to put attributes in alias declarations is that alias declaration are not types. This is a big difference when trying to apply `[[nodiscard]]` in comparison to `[[deprecated]]`. In the case of `[[deprecated]]`, the compiler can issue a warning in the moment that it sees the name in the declaration. For the case of `[[nodiscard]]`, the compiler would have to keep this information additionally because the alias is not a type.

In [P3245R0] I describe two possible solutions to solve the problem presented in the "Motivation" section. However, in St. Louis EWGI voted to pursue the option to allow `[[nodiscard]]` in type aliases. In newer versions of the paper I will focus on this option and maintain the second option just for documentation purposes.

§ 5.1.1 Aliases carry `[[nodiscard]]` information

This option would require to remember if a function was seen with the alias or with the original type. This would mean that if the compiler has seen the function with the return alias, it should issue the warning when the return value is not used.

```
Error foo(int bar);

foo(42); // No warning is issued`

MyError foo(int bar); // Same declaration like above due to Error and
        MyError being the same type

foo(42); // `[[nodiscard]] warning is issued`
```

Because there is already an implementation on how `[[nodiscard]]` could be used in type aliases, the initial goal would be to standarize it in the same way unless we fine good reasons to make it different.

In the following lines, I will expand on some examples and corner cases where it might not be obvious how it should behave.

In case an alias of an alias is introduced, if the alias had `[[nodiscard]]` then the alias of the alias also behaves as `[[nodiscard]]`.

```
struct Error{
    int code;
};

using MyError [[clang::warn_unused_result]] = Error;

using MyOtherError = MyError;

MyOtherError critical_call(){
    return {42};
}

void foo(){
    // The following line will issue a warning because the return type is
        an alias
    // of MyError and the MyError alias has the [[nodiscard]] attribute
    critical_call();
}
```

Compiler explorer

In case of multiple redeclarations, the last one seen wins:

```
struct Error{
    int code;
};

using MyError [[clang::warn_unused_result]] = Error;

MyError critical_call(){
    return {42};
}

void foo(){
    // A warning will be issued because the last seen
    // declaration is of an alias marked [[nodiscard]]
    critical_call();
}

Error critical_call();

int main(){
    // No warning will be issued because the last seen
    // declaration was with a type that was not marked [[nodiscard]]
    critical_call();
}
```

Compiler explorer

§ 5.1.1.1 Voting in St. Louis on this option

Pursue option 1 (attribute on alias)

SF	F	N	A	SA
1	6	3	0	0

Note: Vote based on [P3245R0]

§ 5.1.2 Alias mechanism to introduce a type with different discardability semantics

Another direction could be considered is to introduce a mechanism to introduce a type from another one. In this case, this would mean that `Error` and `MyError` would be two different types, one that is `[[nodiscard]]` and one that is not.

§ 5.1.2.1 Voting in St. Louis on this option

Pursue option 2 (strong types)

SF	F	N	A	SA
2	0	1	4	3

Note: Vote based on [P3245R0]

§ 6 Wording

I did not have time yet to provide a concrete wording for the porposal, but I'm planning to make it available before the next meeting in Wrocław.

§ 7 Acknowledgements

Thanks a lot to everyone that participated in the initial discussion in Mattermost and during the Tokyo meeting, to EWGI for the great experience on presenting my first paper, to Matt Godbolt for Compiler explorer, and to Michael Park for providing the framework to write this paper.

§ 8 References

[P0068R0] Andrew Tomazos. 2015-09-03. Proposal of [[unused]], [[nodiscard]] and [[fallthrough]] attributes.
https://wg21.link/p0068r0
[P0189R1] Andrew Tomazos. 2016-02-29. Wording for [[nodiscard]] attribute.
https://wg21.link/p0189r1
[P2422R0] Ville Voutilainen. 2024-02-09. Remove nodiscard annotations from the standard library specification.
https://wg21.link/p2422r0

[P3201R1] Jonathan Wakely, David Sankel, Darius Neațu. 2024-03-22. LEWG [[nodiscard]] policy.

https://wg21.link/p3201r1

[P3245R0] Xavier Bonaventura. 2024-04-16. Allow `[[nodiscard]]` in type alias declarations.

https://wg21.link/p3245r0